Representative Learning

Autoencoders

Learn dense representations of input data, without any supervision
- Lower dimensions compared to raw input
- Feature detectors
- Used for unsupervised learning
- Generative Models (generate new data from representative data that looks similar to the origin data)

Model forced to learn efficient ways to representing data after being altered (adding noise or dimension reduction). 'Codings' are model learning the identify function under constraints

Raw Image -> [Constraint] -> Raw Image

Constraint is filled with learning artifacts about the raw data

Generative Adversarial Networks

Also, learn without supervision
- Generate faces
- Super resolution of an image
- Image editor (replace background with realistic background)
- Turn sketch into photorealistic image
- Augment data to generate new data (transform audio to image and image to audio)

Generator - works to generate data similar to training data

Discriminator - tells whether the data is fake or real

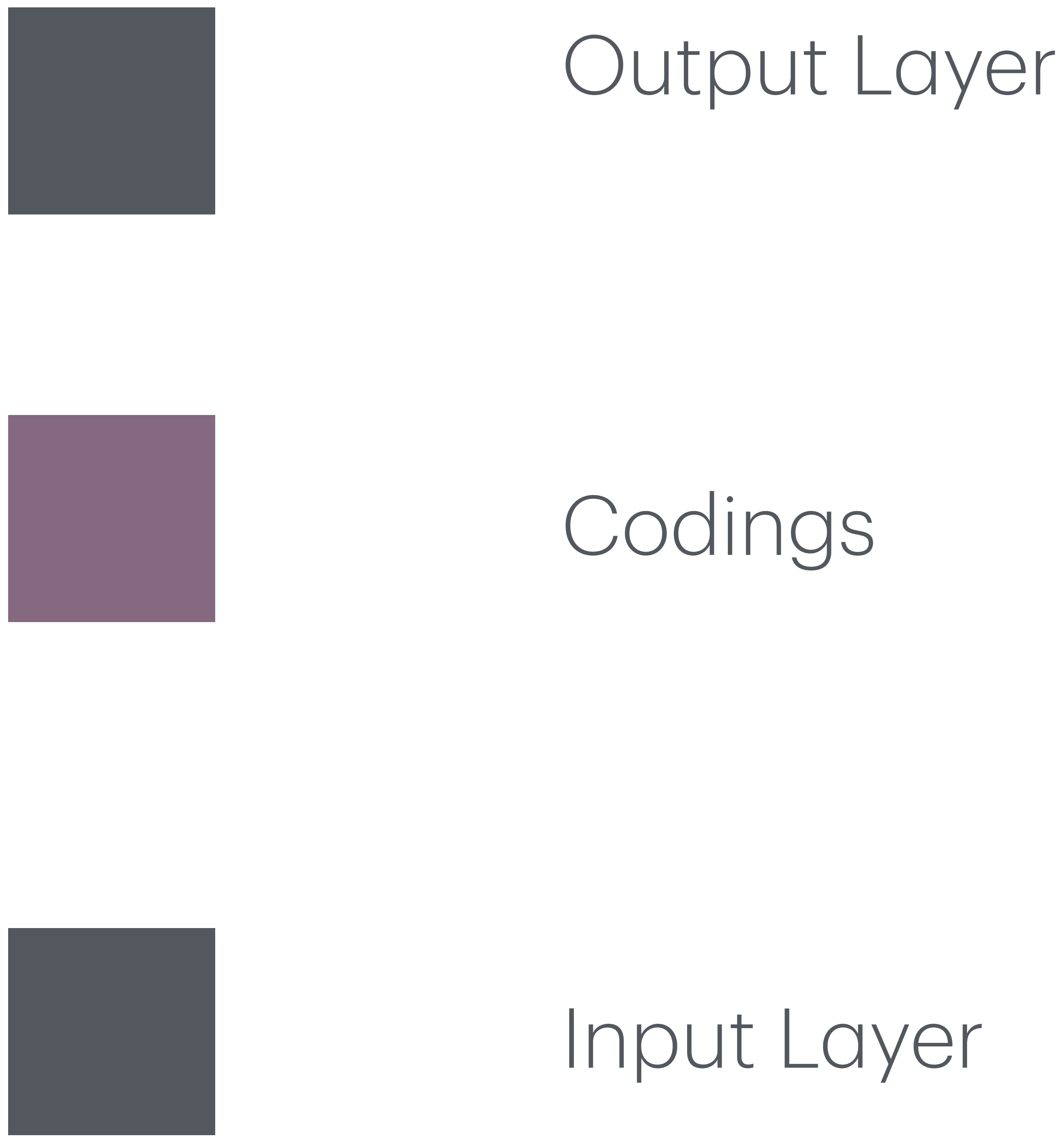
Chess Player

Chess experts recognize patterns which they map to realistic position on a board.

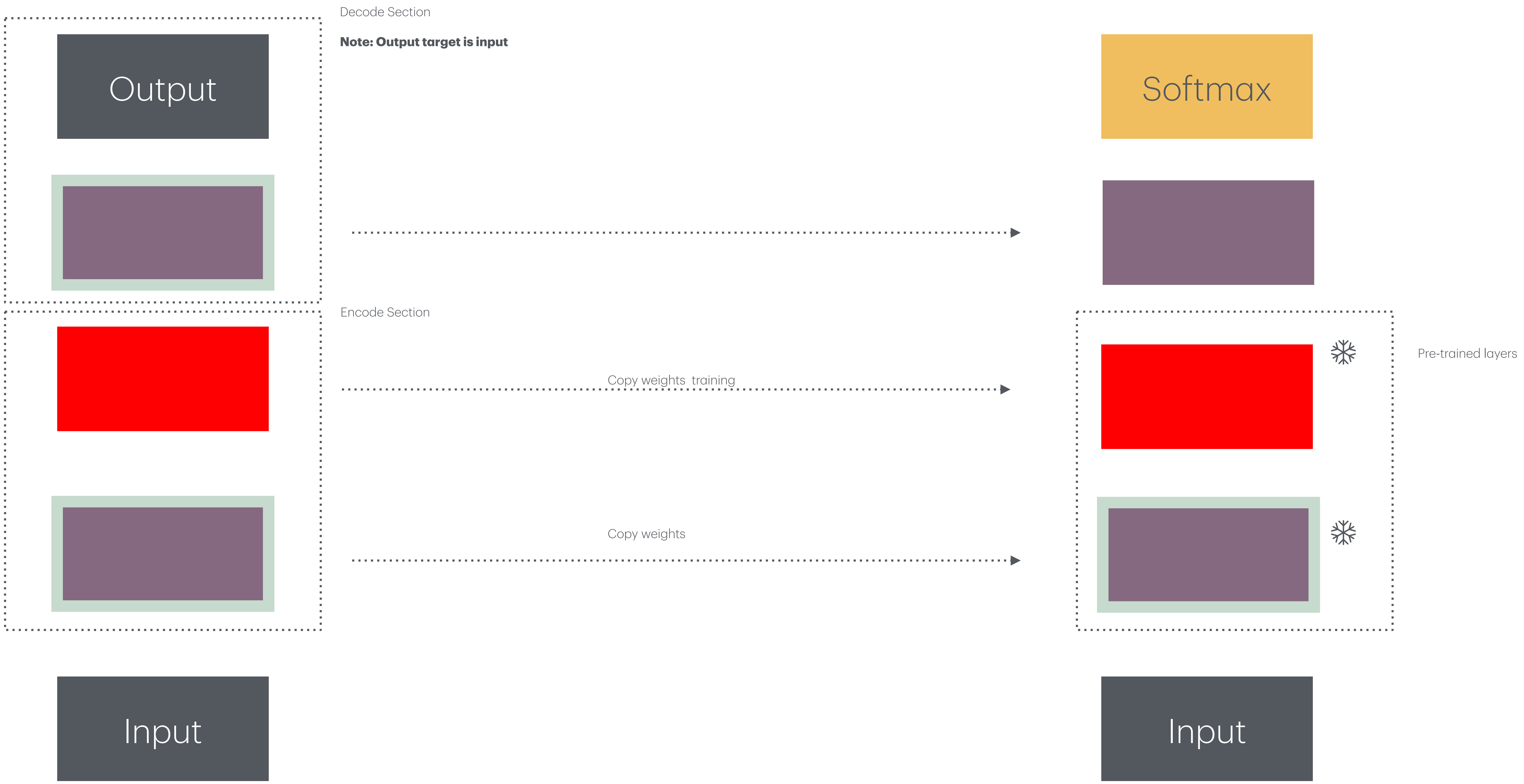
Their memory is finite like every other human but they have trained to recognize patterns which are stored in some 'representative format' within their brain. This format is exactly what the autoencoder does: recognizing patterns and create some data format to represent it. This format is different from the raw input.

AutoEncoder

Deep autoencoder helps learn complex patterns, not to deep(i.e. reconstruction not perfect) in order to generalize to different instances



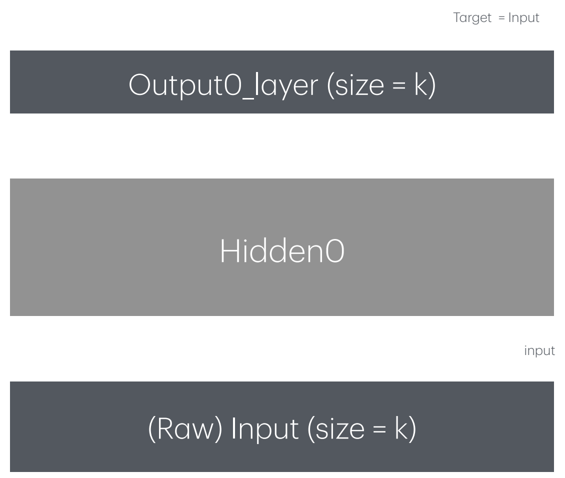
Autoencoder (Unsupervised Pre-training)



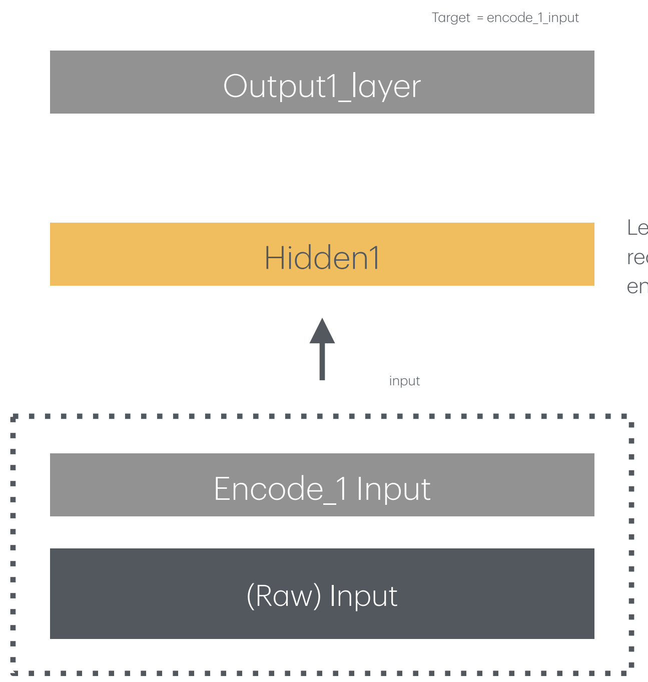
Train on labeled and unlabeled instances

Train on labeled data. Freeze lower stage pre-trained layers (encoder layers)

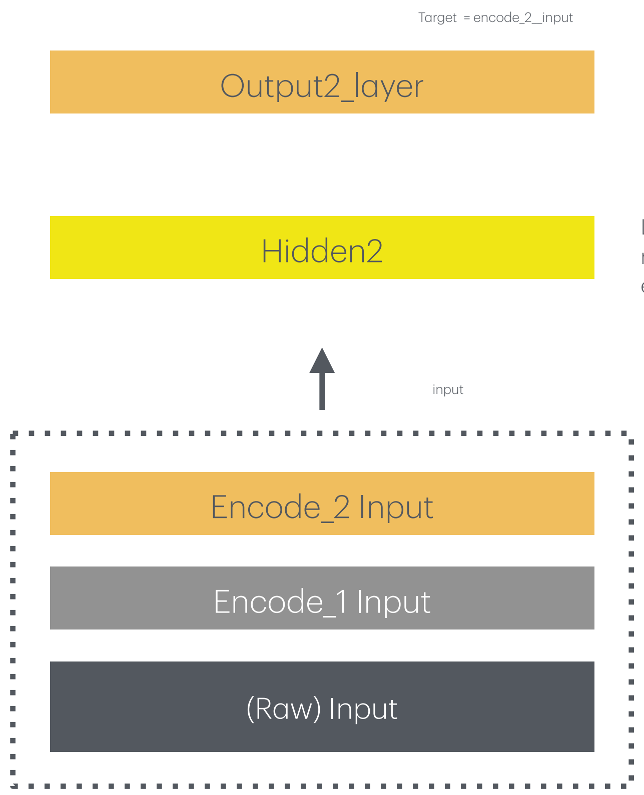
Autoencoder (Training one at a time)



Learn to reconstruct raw input



Learn to reconstruct encoded. 1



Learn to reconstruct encoded. 2

Deep neural network pre-trained using unsupervised method

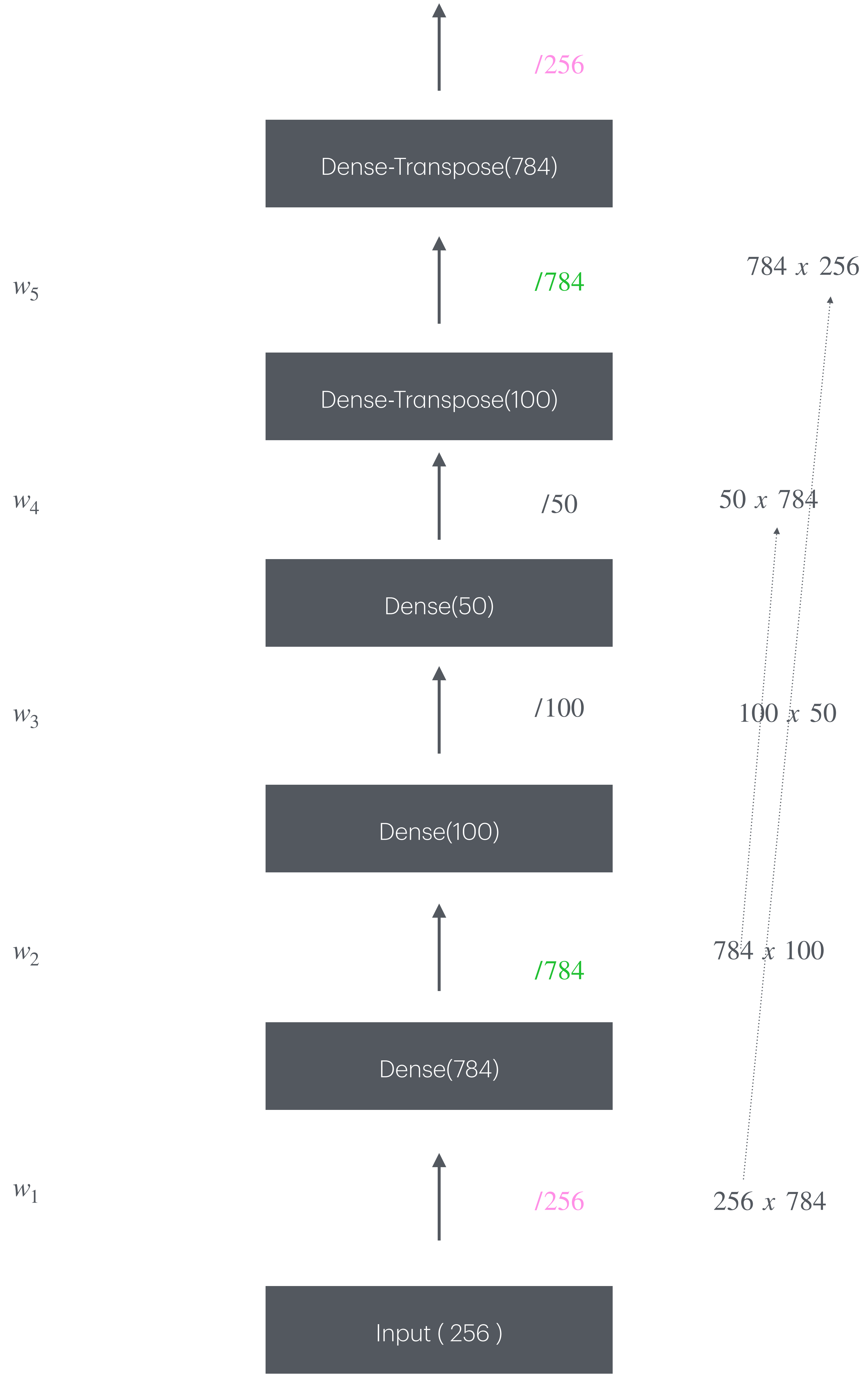
Encoding hidden blocks can trained on limited labeled data and estimated on unlabeled data

Weights Symmetry

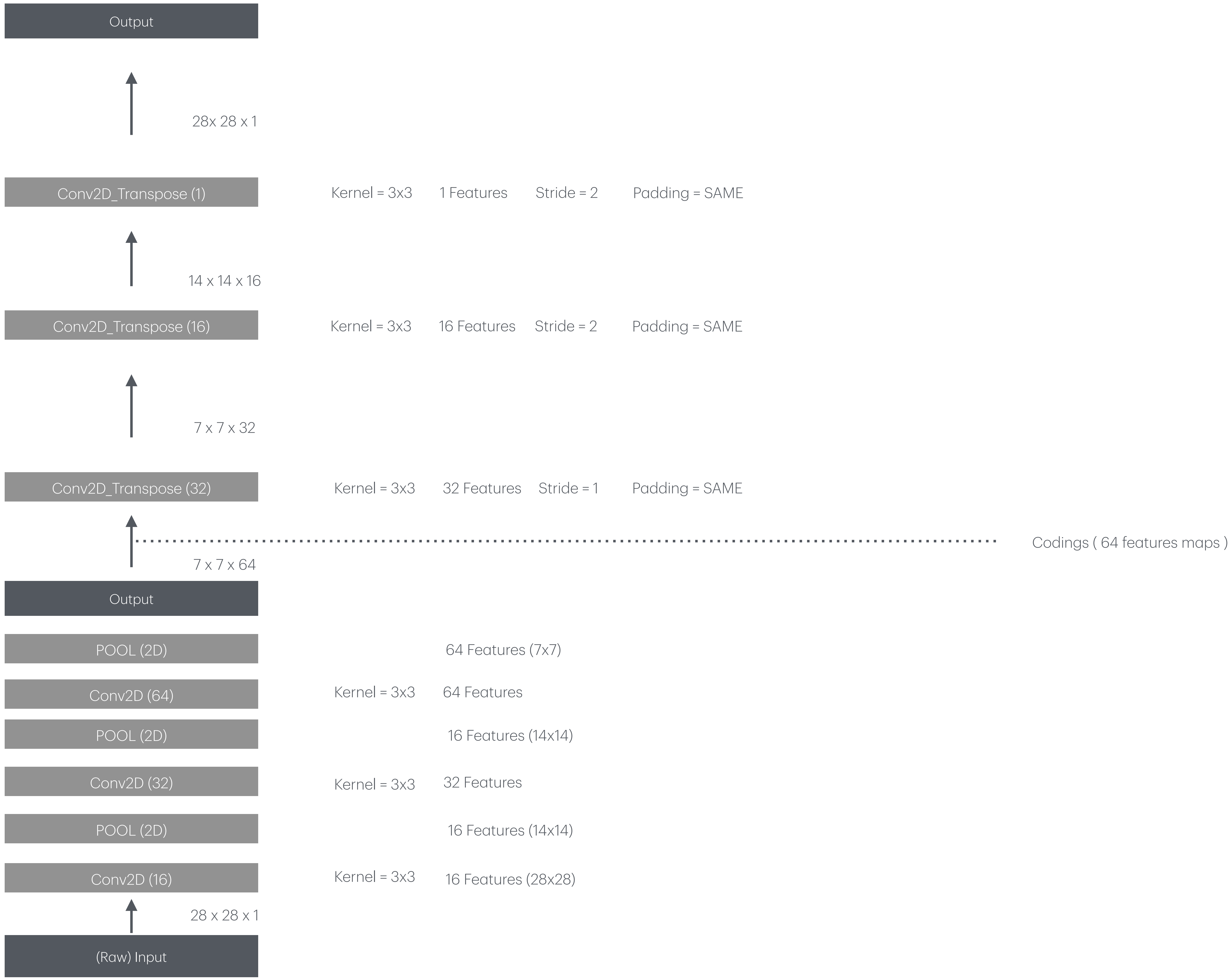
Reduces weights in half (reduces overfitting) and speeds up time

Weights Symmetry

Reduces weights in half (reduces overfitting) and speeds up time

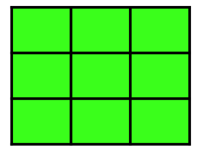


AutoEncoder (Image)

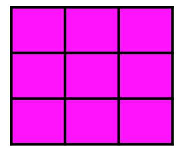


Transpose Convolution 2D

Kernel (3x3)

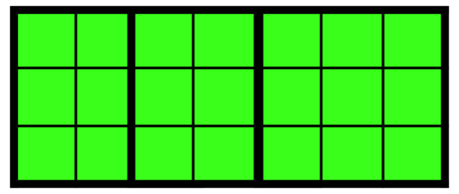


Input Tensor (3x3x64)



Padding = None

Output (7x7x 32) (Single input row observed)

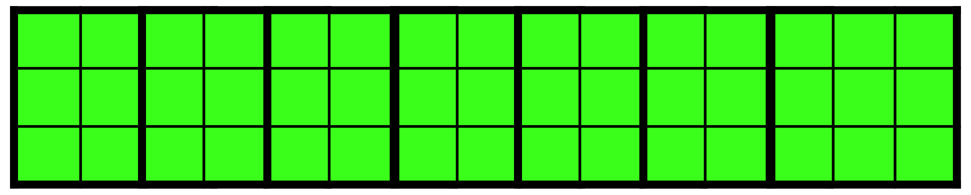


Valid padding formula = $kernel_h + (Input_h - 1) \ * \ stride_h$

---

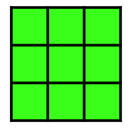
7 * 2 = 14 Total Neurons (padding not needed)

Output (14x14 x 16) (Single input row observed)

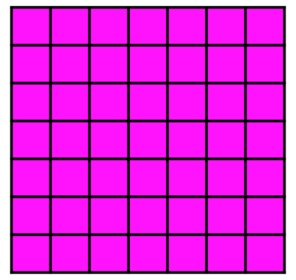


14 x 14

Kernel



Input Tensor (7x7)



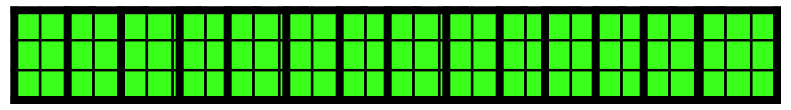
Stride = 2

Padding = SAME

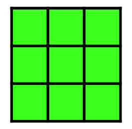
---

14 * 2 = 28 Total Neurons (padding not needed)

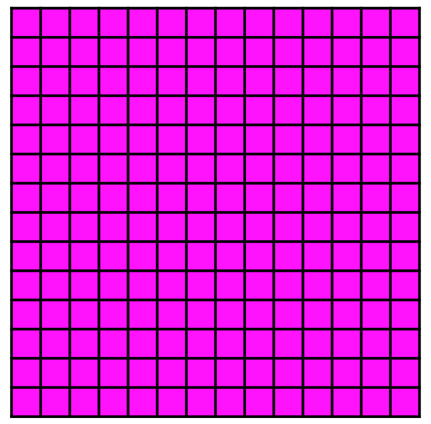
Output (28 x 28x 1) (Single input row observed)



Kernel



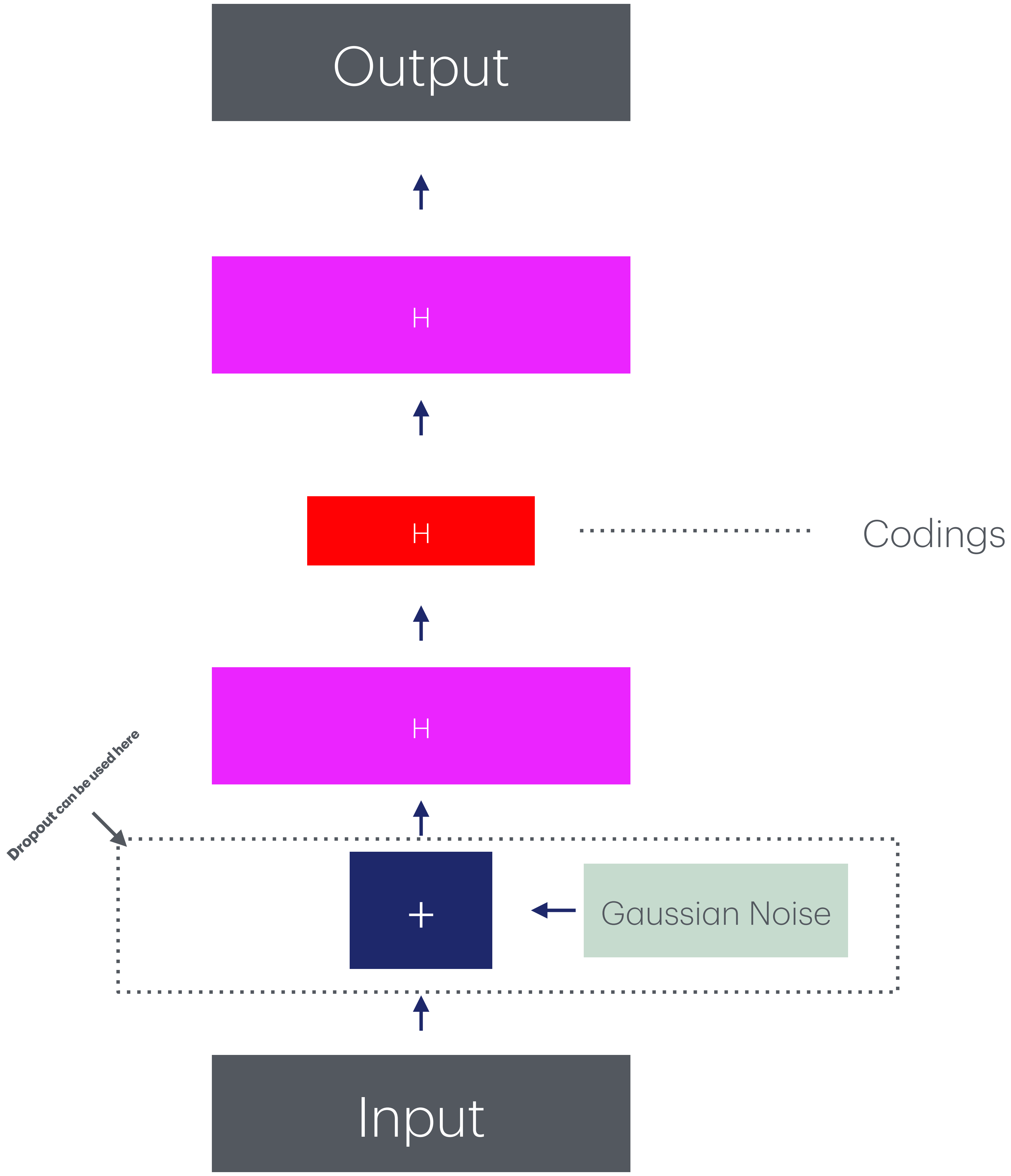
Input Tensor (14 x 14)



Stride = 2

Padding = SAME

Denoise Autoencoder

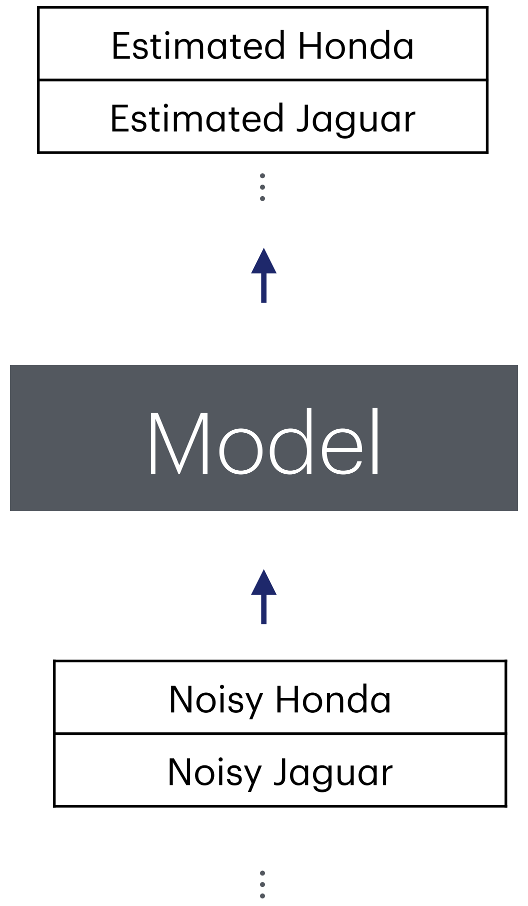


Recover noise free input, model learns remarkable new feats

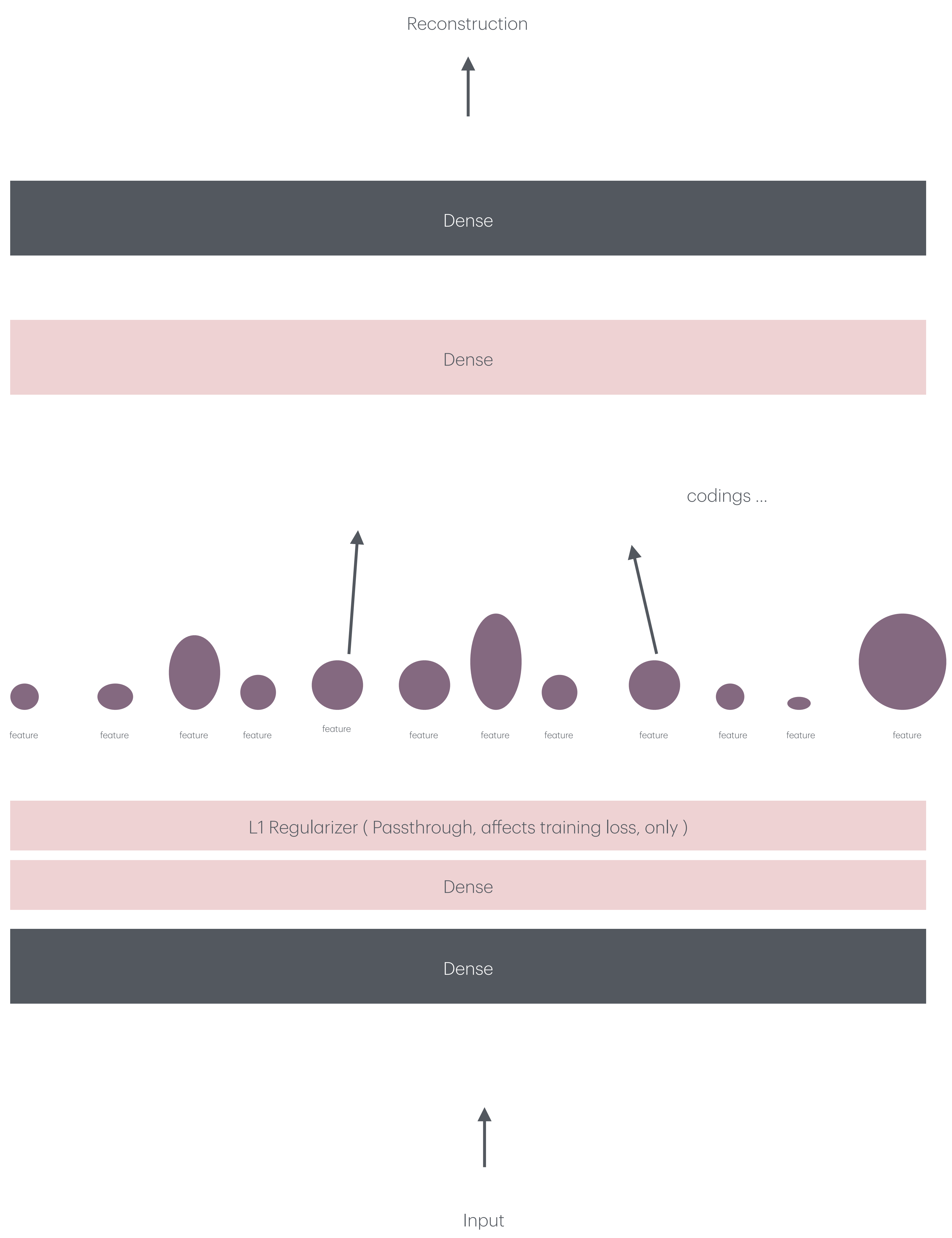
Model classifies damaged images by using learned features learned during training

Example:

"If the image has a sharp curve on the edge where it is blurred, its a plane"



Sparse Autoencoder



Produce codings which use the least amount of neurons

- L1 Regularization. Pulls codings close to zero

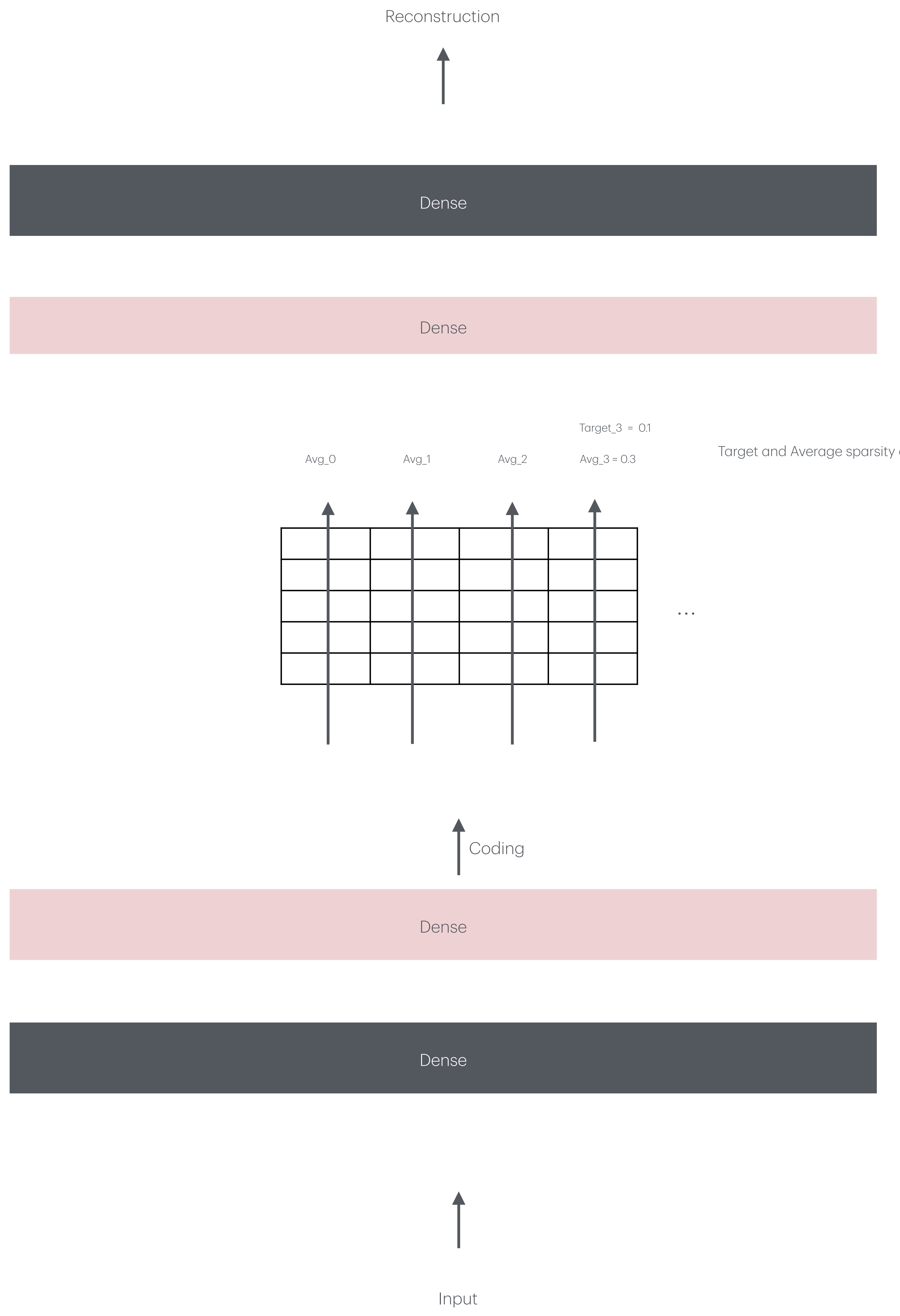
- Reconstruction loss pulls a few neurons high to lower reconstruction loss/error

Analogous to automatic gain control. Where certain channels are weighted higher or lower to keep system stable. Stability would be sustainability of good reconstruction.

L1 regularization penalizes non-important features keeping the most important features (codings)

- L2 pushes works to reduce all codings

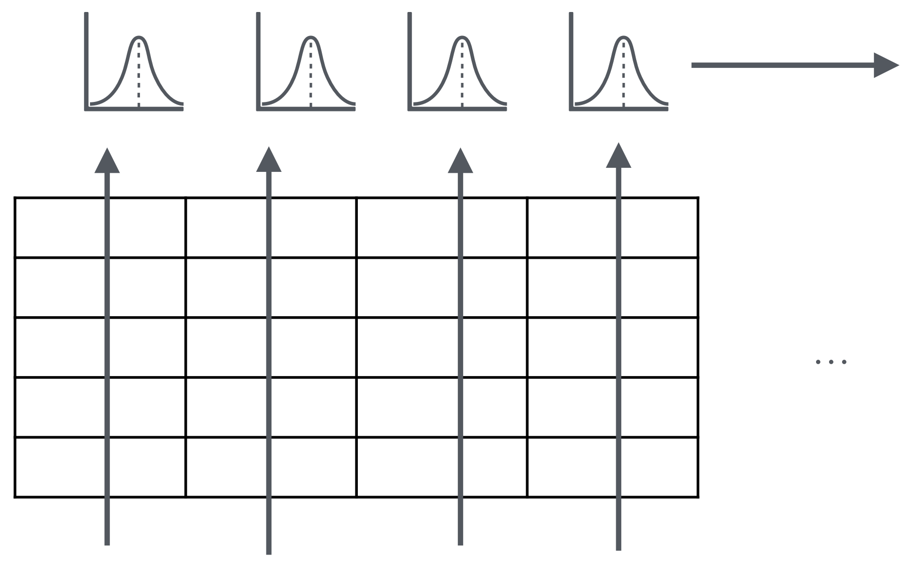
Sparse Autoencoder 2



Target and Average sparsity different by 0.2. Penalize using **KL Divergence**

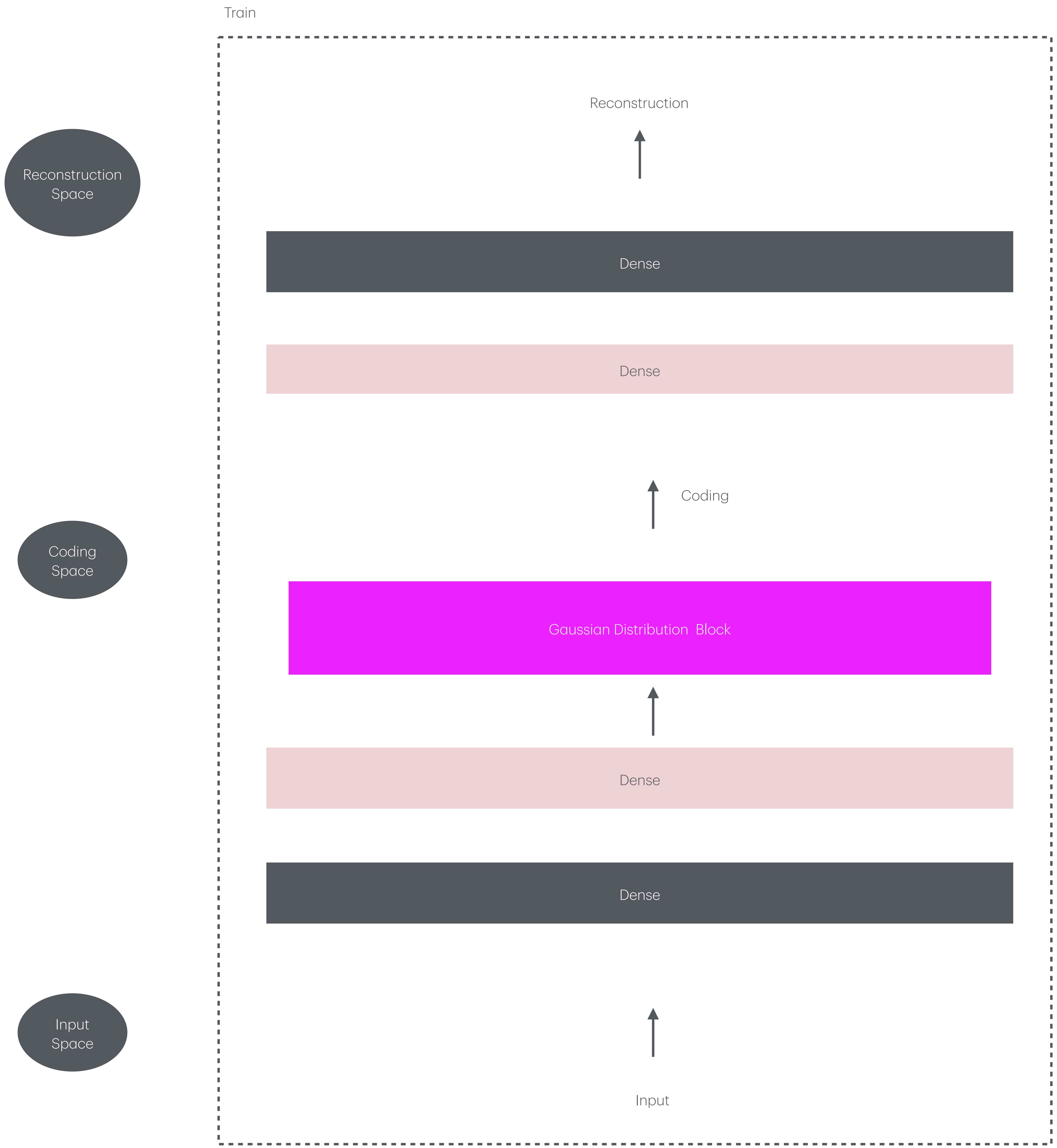
- Penalize neurons that are too active, or not active enough (keep things cool)
- Neurons are influenced to all operate at a steady state.

KL Divergence
Probability of a neuron activating .
q - mean activation over training batch (acquired thru mean of training batch channel)
p - target probability (acquired thru experiment testing ; a constant parameter



Histogram os random measurements over time per neuron channel . Used to find location of most likely output. This is essentially p.

Variational Autoencoder 2

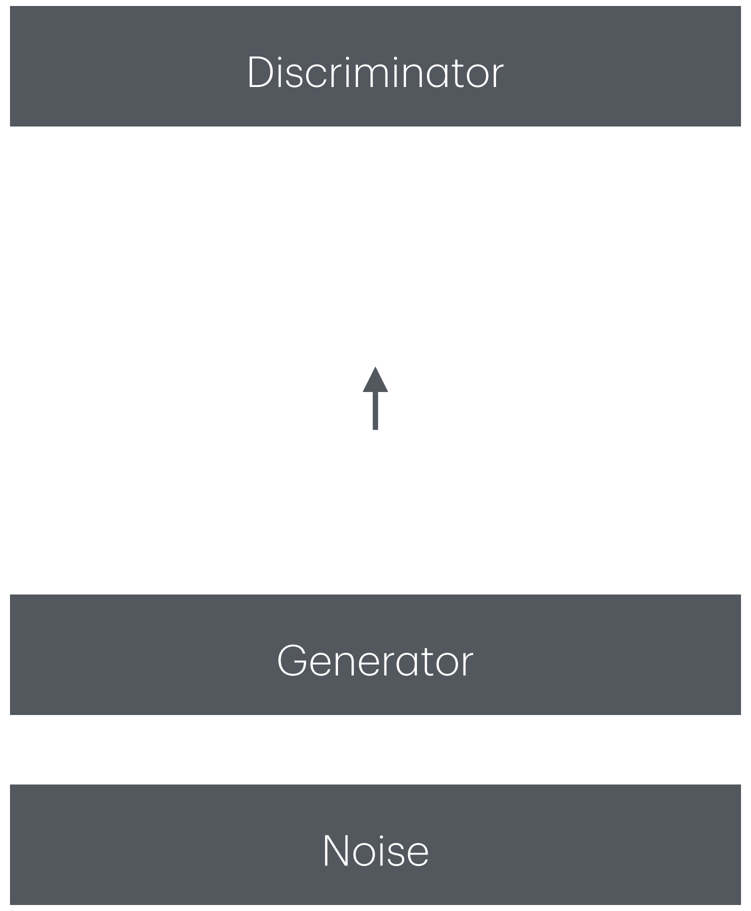


Model 'teaches' Gaussian block to generate statistical codings

During training, model works to map gaussian vectors (i.e. codings) to an image space

Gaussian noise (inside Gaussian Distribution Block) 'constrains' information transmitted to coding layer. Complex features are learned to reconstruct

Generative Adversarial Network

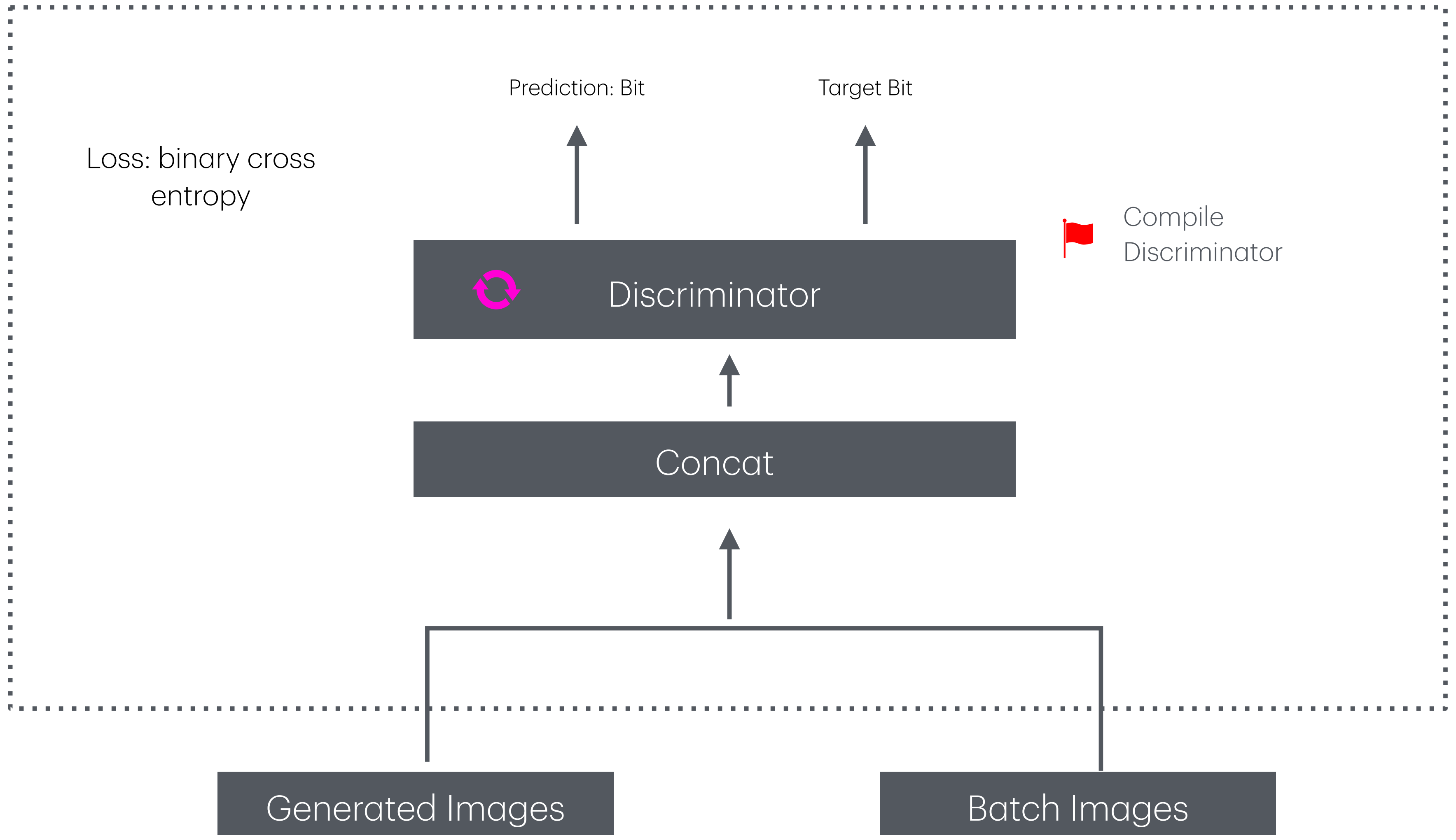


Discriminator Is trained on large dataset of real and fake data (real images, generated images). Learns to classify real and fake images

Generator generates fake data (fake images) and labels them as real. These counterfeit images are passed to the Discriminator. Weights for generator and discriminator are updated to make the future generated (counterfeit) images look as close as possible to real images.

I know what is real, and this is not. You need to tune your variables this way and that way to make better generative data (make better counterfeit bills)

Training 1



Training 2